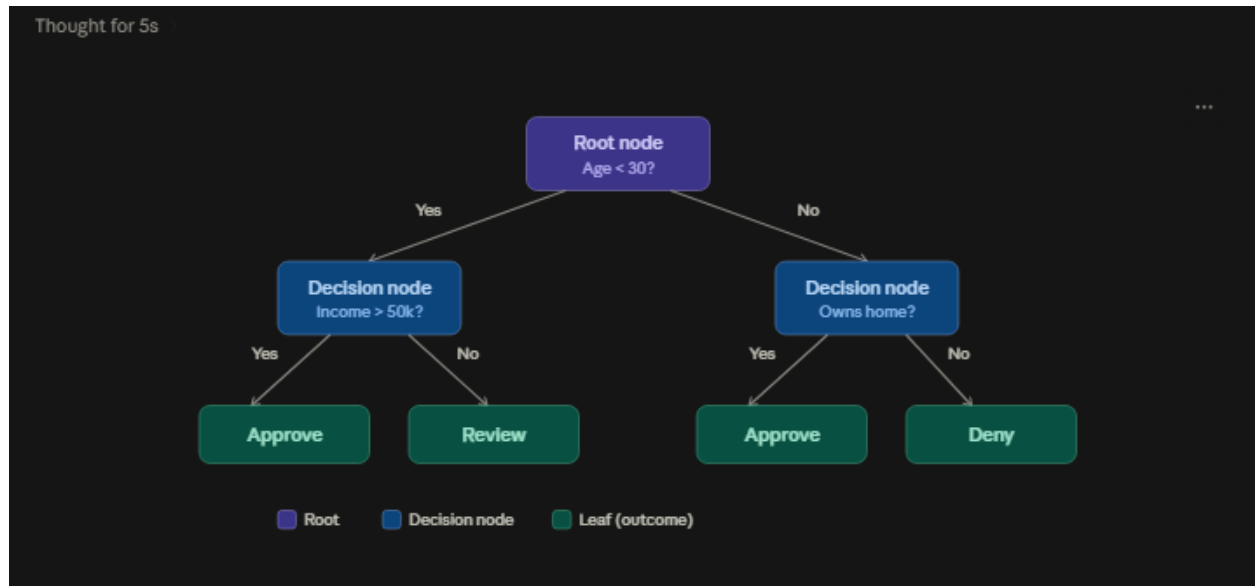


Decision Tree

A decision tree is a supervised learning algorithm used for both classification and regression tasks.



Root node — the first split, based on the feature that best separates the data (e.g. by information gain or Gini impurity)

Decision nodes — internal nodes that keep splitting the data on further conditions

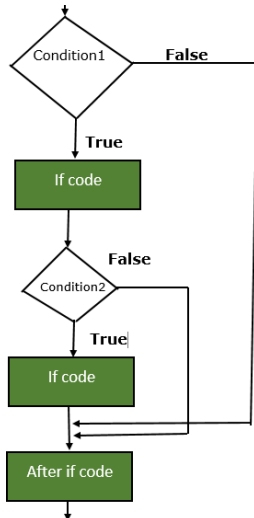
Leaves — the final outcome/prediction once no more splitting happens

How Decision Tree Works:

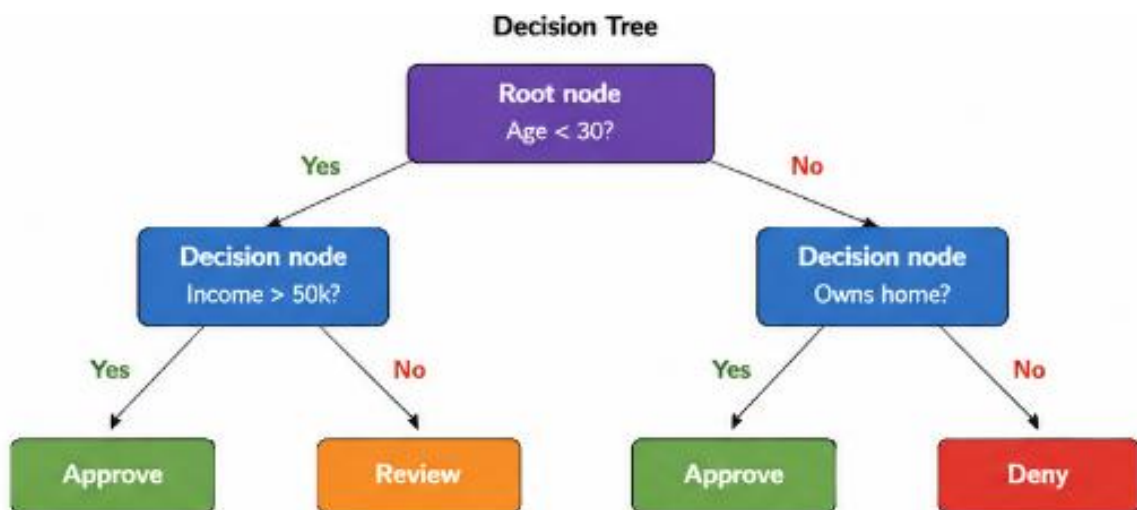
- Begins with a dataset having feature and target.
- Determine the best feature through which data is split
- Split data into subsets containing correct value for best feature
- Recursively generate new tree nodes using the subset of data created from step 3.

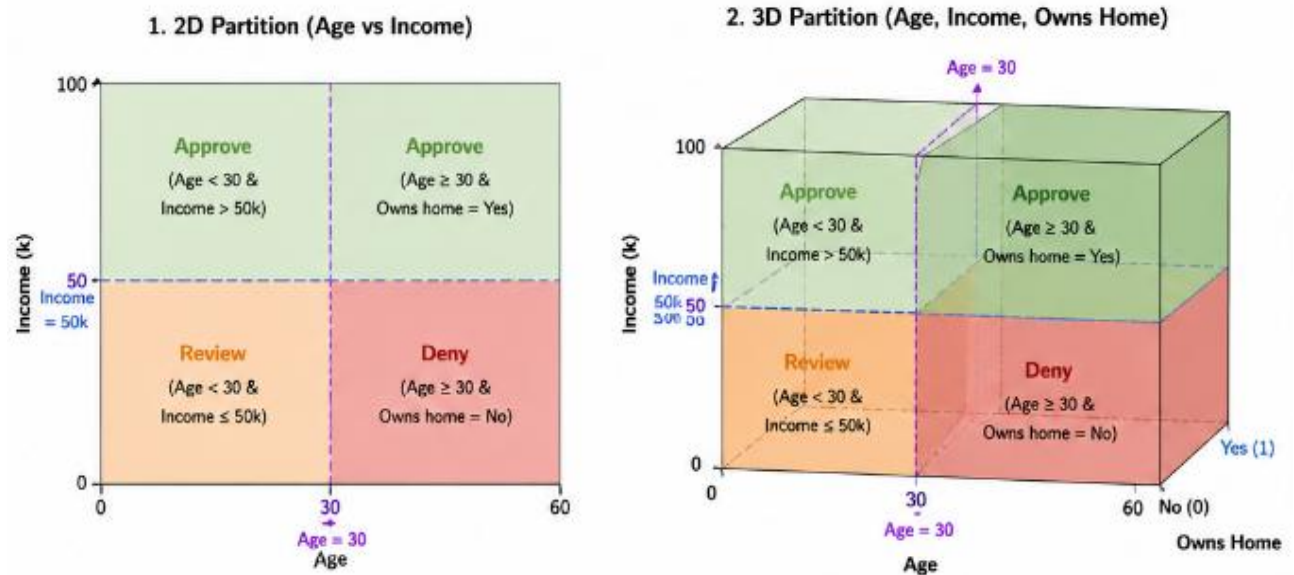
Key Insights:

Programmatically - A decision tree is an algorithm that is similar to a big structure of nested if-else.



Mathematically – Decision trees use hyperplanes which run parallel to one of the axes to cut the coordinate system into hyper cuboids.





Entropy:

Entropy is a measure of impurity or randomness in a dataset.

Formula

$$Entropy(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

Where:

- S = dataset/node
- p_i = probability (proportion) of class i
- $\log_2$ = logarithm with base 2

Let's suppose we have data with 2 class labels 'yes' and 'no' then the entropy will be:

$$E(S) = - p_{yes} \log_2(p_{yes}) - p_{no} \log_2(p_{no})$$

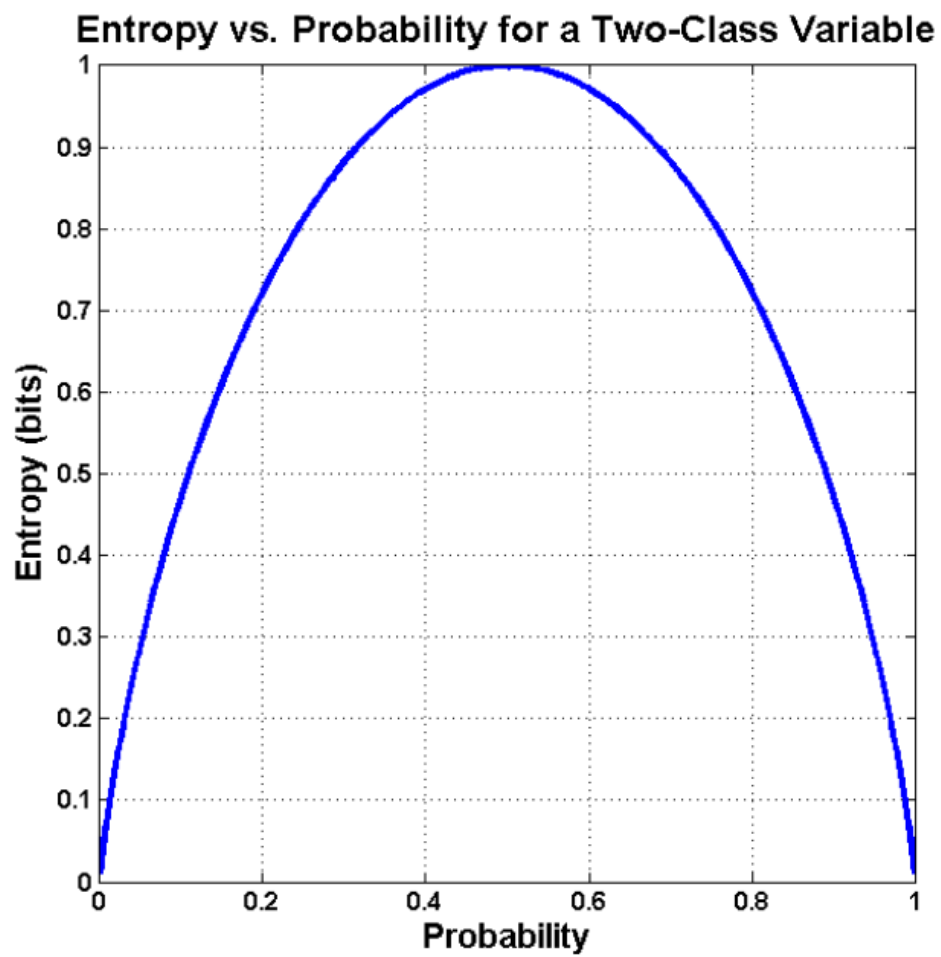
If more than 2 labels then we add the label like

$$E(S) = - p_{\text{yes}} \log_2(p_{\text{yes}}) - p_{\text{no}} \log_2(p_{\text{no}}) - p_{\text{label}_3} \log_2(p_{\text{label}_3})$$

Key Points:

- If there is more uncertainty, entropy will be higher. In other words, higher uncertainty means lower information, while lower uncertainty means higher information.
- Entropy will be between 0-1, 0 being minimum and 1 being maximum for **2 class problem**.
- Entropy can be greater than 1, 0 being the minimum for **more than 2 class** i.e.
0 - (more than one)
- We can use both **$\log_2$** or **$\log_e$** for calculating entropy.

Entropy vs Probability Graph (2 class variable)



Information Gain

Information Gain (IG) measures how much uncertainty is reduced after splitting the data using a feature.

Information Gain tells us how useful a feature is for making a decision.

The formula is:

$$IG = \text{Entropy}_{(\text{parent})} - \text{Weighted Entropy}_{(\text{children})}$$

High Information Gain = Large reduction in entropy

Calculating Information Gain:

Let's suppose we have a dataset:

ID	Weather	Play
1	Sunny	No
2	Sunny	No
3	Sunny	Yes
4	Rainy	Yes
5	Rainy	Yes
6	Rainy	No
7	Cloudy	Yes
8	Cloudy	Yes

To calculate Information Gain,

First, we calculate the entropy of parent:

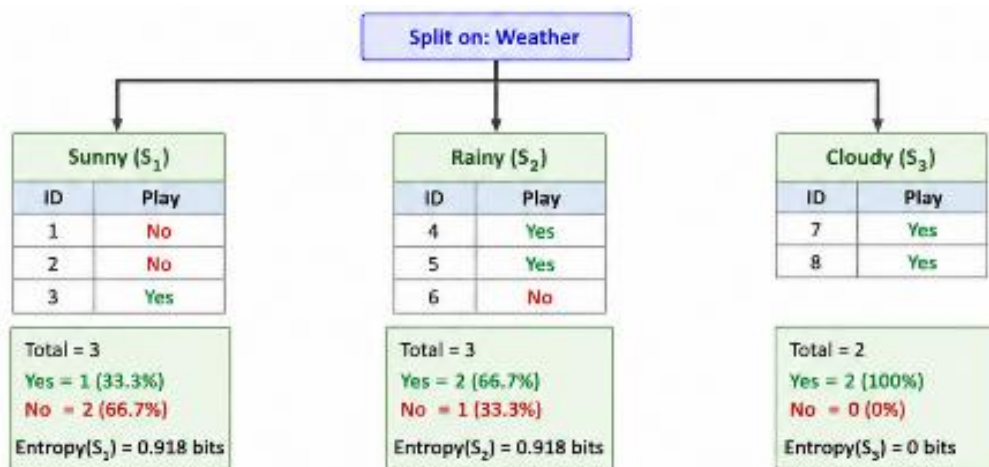
Probability of 'yes' (P_{yes}) = $5/8 = 0.625$

Probability of 'no' (P_{no}) = $3/8 = 0.375$

$$\begin{aligned}\text{Entropy}_{(\text{parent})} &= -p_{\text{yes}} \log_2(P_{\text{yes}}) - p_{\text{no}} \log_2(p_{\text{no}}) \\ &= (0.625) \log_2(0.625) - (0.375) \log_2(0.375) \\ &= 0.954\end{aligned}$$

So, the entropy before splitting is 0.954.

Now,



Splitting by weather:

1) Sunny:

Yes = 1

No = 2

$$\begin{aligned}\text{Entropy of Sunny } (E_{\text{sunny}}) &= -(1/3) \log_2(1/3) - 2/3 \log_2(2/3) \\ &= 0.918\end{aligned}$$

2) Cloudy:

Yes = 2

No = 0

Since this group has 100% yes, the entropy will be 0.

3) Rainy:

Yes = 2

No = 1

$$\begin{aligned}\text{Entropy of Rainy (E}_{\text{rainy}}) &= -(2/3) \log_2(2/3) - 1/3 \log_2(1/3) \\ &= 0.918\end{aligned}$$

Calculate Weighted Entropy for children:

$$\begin{aligned}\text{Weighted Entropy}_{\text{children}} &= 3/8 * (0.918) + 3/8 * (0.918) + 2/8 * 0 \\ &= 0.689\end{aligned}$$

Now,

Calculating Information Gain:

$$\begin{aligned}\text{Information Gain (IG)} &= \text{Entropy}_{\text{parent}} - \text{Weighted Entropy}_{\text{children}} \\ &= 0.954 - 0.689 \\ &= 0.265\end{aligned}$$

Thus, The **Weather** feature gives us an Information Gain of approximately **0.265 bits**.

Gini Impurity

Gini Impurity is another way that decision trees measure how mixed or impure a node is. It measures the probability of incorrectly classifying a randomly chosen sample if it were labeled according to the class distribution of that node.

Formula

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

For two classes:

$$Gini = 1 - (p_{Yes}^2 + p_{No}^2)$$

Example of calculating Gini Impurity:

Example 3 - Dataset

Salary	Age	Purchase
20000	21	Yes
10000	45	No
60000	27	Yes
15000	31	No
12000	18	No

$$G = 1 - (P_y^2 + P_n^2)$$
$$G = 1 - (4/25 + 9/25)$$
$$G = 0.48$$

Salary	Age	Purchase
34000	31	No
15000	25	No
69000	57	Yes
25000	21	No
32000	28	No

$$G = 1 - (P_y^2 + P_n^2)$$
$$G = 1 - (1/25 + 16/25)$$
$$G = 0.32$$

Information gain using Gini impurity:

$$IG_{\text{Gini}} = \text{Gini}_{\text{parent}} - \text{Weighted Gini}_{\text{split}}$$

Why Gini is often preferred

The biggest practical advantage is simplicity and speed.

Entropy requires logarithms:

$$E(S) = -p_{\text{yes}} \log_2(P_{\text{yes}}) - p_{\text{no}} \log_2(p_{\text{no}})$$

Gini only requires squares:

$$G = 1 - (P_{\text{yes}}^2 + P_{\text{no}}^2)$$

So, Gini is generally computationally cheaper, especially when a decision tree has many possible splits.

When should you use Gini?

- You want a fast decision tree
- You're working with a large dataset
- You don't have a specific reason to use entropy
- You're using algorithms such as CART

4. When should you use Entropy?

- You specifically want an information-theory-based measure
- You want to understand Information Gain mathematically
- You're studying algorithms such as ID3
- You want to compare splits based on reduction in uncertainty